

UNIVERSITÀ DEGLI STUDI

RELAZIONE DI PROGETTO

“WebTennis” - Web App per la Gestione Sportiva del Circolo Tennis

LORENZO VANNUCCI – MATR. 370003

Corso di Programmazione Web e Mobile
Anno Accademico 2025/2026

1.1 L'idea alla base del progetto

Per il progetto dell'esame di Programmazione Web e Mobile, ho deciso di sviluppare una piattaforma web dedicata alla gestione sportiva e organizzativa di un circolo di tennis. L'ispirazione per questo progetto mi è venuta osservando come, in molti piccoli circoli sportivi della mia zona, l'organizzazione dei match e delle classifiche venga ancora gestita in modo molto "analogico", utilizzando fogli di carta appesi in bacheca o gruppi WhatsApp spesso confusionari.

Il mio obiettivo principale era quindi quello di creare un'applicazione web moderna e intuitiva che permettesse ai soci di avere un punto di riferimento centrale. Volevo un sistema dove ognuno potesse registrarsi, controllare le proprie statistiche, vedere le classifiche in tempo reale e, soprattutto, inserire i risultati delle partite in autonomia direttamente dal proprio smartphone subito dopo aver giocato.

1.2 I requisiti che mi sono posto

Prima di iniziare a scrivere il codice, ho buttato giù una lista delle funzionalità fondamentali che volevo implementare per rendere il progetto completo e realistico:

- **Gestione degli utenti:** Il sito deve avere una parte pubblica (vetrina del circolo) e una privata. Per accedere alle classifiche e inserire i risultati, l'utente deve registrarsi e fare il login.
- **Profilo personale interattivo:** Ogni giocatore doveva avere una sua pagina con le statistiche aggiornate (partite vinte, perse, punteggio totale accumulato).
- **Classifiche automatiche:** Un sistema che calcola e ordina i soci in base ai punteggi acquisiti, separando la classifica maschile da quella femminile per un po' di competizione.
- **Pannello per l'Amministratore:** Mi sono reso conto che far inserire i punteggi liberamente ai soci poteva portare ad errori. Quindi ho implementato un ruolo di "Amministratore" (lo staff del circolo) che ha il potere di visualizzare lo storico di tutti i match del club e, se necessario, correggere i risultati inseriti in modo sbagliato.

1.3 Le scelte architetturali e le tecnologie usate

Come richiesto dalle specifiche del corso, ho strutturato l'applicazione seguendo il pattern Client-Server a tre livelli. Non ho usato framework troppo pesanti o librerie estranee al corso per il frontend, in modo da mantenere il più semplice possibile la struttura:

1. **Frontend (Il Client):** Ho scritto l'interfaccia interamente in HTML e CSS. Per la dinamicità della pagina (chiamate asincrone, validazione dei form, grafici), ho usato JavaScript puro, integrando le API di Google Charts per disegnare i grafici delle statistiche.
2. **Backend (Il Server):** Ho scelto Node.js con il framework Express.js. È la tecnologia con cui mi sono trovato meglio a lezione per creare rapidamente delle API RESTful.
3. **Database (I Dati):** Per salvare i dati in modo coerente e sicuro ho usato un database relazionale, MySQL. Nel backend, ho usato `mysql2` per interfacciarmi al database usando le *Promises* in JavaScript.

2.1 Perché ho scelto i JSON Web Token (JWT)

Una delle parti su cui ho speso più tempo è stata capire come gestire il login degli utenti in modo sicuro. All'inizio pensavo di usare le classiche sessioni salvate nella memoria del server, ma poi ho deciso di implementare i **JSON Web Token (JWT)**.

Usare i JWT mi ha permesso di creare un server *stateless*: il server non si deve ricordare chi è loggato memorizzando dati nella sua RAM. Quando un utente fa il login e la password è corretta, il server crea questo “gettone” (il JWT) e lo firma con una chiave segreta. Dentro questo gettone ci ho messo solo le informazioni essenziali:

- **userId**: l'ID univoco dell'utente nel database.
- **isAdmin**: un valore booleano (0 o 1) per capire al volo se chi sta facendo la richiesta è un socio normale o fa parte dello staff.

2.2 Il problema della sicurezza: dove salvo il token?

Dopo aver generato il token, dovevo decidere come farlo arrivare al client e come conservarlo. La soluzione più facile sarebbe stata metterlo nel `localStorage` del browser, ma leggendo online e rivedendo gli appunti ho capito che è molto vulnerabile agli attacchi XSS (se qualcuno riesce a iniettare un po' di JavaScript malevolo nella pagina, può rubare il token a tutti).

Quindi ho optato per salvare il token in un **Cookie HTTP-Only**. In Node.js, quando rispondo al login con successo, imposto il cookie in questo modo:

- **httpOnly**: `true`: Così il cookie non può mai essere letto dal codice JavaScript nel frontend. È il browser che lo gestisce automaticamente.
- **sameSite**: `'strict'`: Questo è fondamentale per proteggere l'app dagli attacchi CSRF (Cross-Site Request Forgery), impedendo che il cookie venga inviato se la richiesta parte da un altro sito web.

2.3 I Middleware di protezione

Per assicurarmi che chi non è loggato non possa fare danni, ho scritto un paio di funzioni “middleware” in Express. Praticamente, ogni volta che un utente prova ad accedere a una pagina privata o a una rotta API sensibile, Express esegue prima questo middleware.

- Se l'utente cerca di caricare una pagina HTML come `giocatore.html` ma non ha il cookie valido, il middleware intercetta la richiesta e fa un redirect forzato alla pagina `denied.html`.
- Se invece la richiesta è una chiamata AJAX (una `fetch` dal frontend alle API REST), il middleware non fa un redirect (perché romperebbe il JSON), ma risponde direttamente con uno *status code* 401 (Non autorizzato).

Per tenere il progetto ordinato, ho diviso nettamente la parte pubblica da quella privata.

3.1 La parte pubblica del sito

Questa è la vetrina del circolo tennis. È accessibile a chiunque e serve per attirare nuovi iscritti.

1. **Home Page** (`index.html`): Un'immagine di copertina, la filosofia del circolo, le attività principali (Scuola Tennis, Tornei) e le card con le foto dei membri dello staff.
2. **Abbonati** (`abbonati.html`): Una pagina con le pricing cards (tessere Socio Base, Socio Agonista, ecc.) per far vedere i costi.
3. **Contatti** (`contatti.html`): Un form di contatto fittizio e una mappa interattiva inserita tramite un `iframe` di Google Maps.
4. **Pagine di Autenticazione**: `login.html` e `registrati.html`. In quest'ultima ho inserito dei controlli HTML sui campi (come l'email obbligatoria) per avere una prima validazione lato client.

3.2 L'area privata (solo per i soci)

Una volta fatto il login, si sbloccano le pagine operative:

1. **La Classifica** (`ranking.html`): Questa pagina carica dal database tutti gli utenti ordinandoli per punteggio. Ho creato due tabelle (Maschile e Femminile). Se un utente normale visita questa pagina, vede solo le classifiche. Se ci entra un amministratore, compare un pannello extra in basso che mostra lo "Storico Incontri" globale, da cui può supervisionare chi ha giocato contro chi in tutto il circolo.
2. **La Scheda Personale** (`giocatore.html`): Questa è la pagina più complessa del frontend. Mostra i dati anagrafici, un grafico a torta con le vittorie e le sconfitte, e lo storico personale dei match. Qui i giocatori possono inserire un nuovo risultato, mentre gli amministratori (grazie a dei controlli JS che leggono i privilegi dell'utente) vedono comparire il tasto per **modificare** i punteggi vecchi.

4.1 Come ho strutturato le tabelle

Per la base di dati del progetto, ho usato **MySQL**. Ho cercato di applicare le regole di normalizzazione viste a lezione per evitare dati duplicati. Alla fine ho organizzato il database `circolo_tennis` in tre tabelle principali.

- **Utenti:** È la tabella anagrafica principale. Ho messo il campo `email` come `UNIQUE` così il database stesso mi impedisce di avere due iscritti con la stessa mail (cosa che gestisco poi nel backend bloccando la registrazione). Ho aggiunto il campo `is_admin` di tipo booleano (`TINYINT`) per i privilegi.
- **Classifiche:** Invece di mettere i punti e le statistiche sportive direttamente nella tabella Utenti, ho preferito separare le cose logicamente. C'è una relazione **1-a-1** stretta. La chiave primaria di Classifiche (`id_giocatore`) è anche una Foreign Key verso l'id dell'utente. Se cancello un utente, la clausola `ON DELETE CASCADE` cancella in automatico le sue statistiche.
- **Partite:** Questa tabella memorizza lo storico dei match. Ogni record ha ben tre foreign key: chi è il giocatore 1, chi è il giocatore 2, e chi dei due ha vinto. Ho aggiunto i campi per i set (fino a 3 set).

4.2 Il codice SQL per creare il database

Questo è lo script DDL per generare lo schema di base:

```
1 CREATE TABLE Utenti (  
2     id_utente INT AUTO_INCREMENT PRIMARY KEY,  
3     nome VARCHAR(50) NOT NULL,  
4     cognome VARCHAR(50) NOT NULL,  
5     email VARCHAR(100) NOT NULL UNIQUE,  
6     password VARCHAR(255) NOT NULL,  
7     genere ENUM('M', 'F') NOT NULL,  
8     is_admin TINYINT(1) DEFAULT 0 NOT NULL -- 0 socio, 1 Admin  
9 );  
10  
11 CREATE TABLE Classifiche (  
12     id_giocatore INT PRIMARY KEY,  
13     punteggio_attuale INT DEFAULT 0 NOT NULL,  
14     posizione_classifica INT NULL,  
15     livello VARCHAR(30) DEFAULT 'Principiante' NOT NULL,  
16     partite_vinte INT DEFAULT 0 NOT NULL,
```



```

17     partite_perse INT DEFAULT 0 NOT NULL,
18     FOREIGN KEY (id_giocatore) REFERENCES Utenti(id_utente) ON DELETE
    CASCADE
19 );
20
21 CREATE TABLE Partite (
22     id_partita INT AUTO_INCREMENT PRIMARY KEY,
23     id_giocatore1 INT NOT NULL,
24     id_giocatore2 INT NOT NULL,
25     set1_g1 INT NOT NULL,
26     set1_g2 INT NOT NULL,
27     set2_g1 INT NOT NULL,
28     set2_g2 INT NOT NULL,
29     set3_g1 INT NULL,
30     set3_g2 INT NULL,
31     id_vincitore INT NOT NULL,
32     data_match TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
33     FOREIGN KEY (id_giocatore1) REFERENCES Utenti(id_utente) ON DELETE
    CASCADE,
34     FOREIGN KEY (id_giocatore2) REFERENCES Utenti(id_utente) ON DELETE
    CASCADE,
35     FOREIGN KEY (id_vincitore) REFERENCES Utenti(id_utente) ON DELETE
    CASCADE
36 );

```

Listing 4.1: Lo script setup.sql che ho usato per creare il DB

4.3 L'ottimizzazione del Connection Pool

Nel codice Node.js, invece di usare `mysql.createConnection()` che apre e chiude una singola connessione per ogni richiesta (rallentando un sacco il server se ci sono più utenti), ho configurato un **Connection Pool**. Così il server tiene aperte un massimo di 10 connessioni in background e le riutilizza in modo efficiente per gestire le richieste asincrone dei client. Inoltre, per proteggere il database dagli attacchi di tipo SQL Injection, ho usato sempre i *Prepared Statements* (passando i parametri tramite il carattere ?), lasciando che sia la libreria `mysql2` a sanitizzare gli input dell'utente.

5.1 Le mie scelte estetiche

Non volevo che l'applicazione sembrasse un progetto “scolastico” grezzo, quindi ho perso parecchio tempo a curare la User Interface (UI). Ho scelto una palette di colori che richiamasse subito il mondo del tennis:

- Un **verde scuro** (#053C26) per i bottoni principali e l'intestazione, simile al colore dell'erba di Wimbledon.
- Un **arancione acceso** (#CB5A3E) per richiamare la terra rossa dei campi, usato per i bottoni al passaggio del mouse (:hover) o per far risaltare elementi importanti.
- Uno sfondo **grigio/sabbia molto chiaro** (#F4F2EE) per non affaticare gli occhi degli utenti, al posto del solito bianco puro.

Per i font, ho incluso da Google Fonts il “Poppins” per i titoli (perché è molto rotondo e moderno) e il “Lato” per i testi lunghi.

5.2 CSS Flexbox e Grid

Ho usato **CSS Grid** per creare i layout a griglia. Ad esempio, nella pagina degli abbonamenti, ho affiancato le tre tessere dei prezzi semplicemente scrivendo:

```
1 .pricing-grid {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4   gap: 40px;  
5 }
```

Listing 5.1: La mia regola per la griglia degli abbonamenti

Invece, per centrare verticalmente il testo dentro i bottoni o allineare i punteggi nei form, ho usato prevalentemente **Flexbox**.

5.3 Il Responsive Design per gli Smartphone

Ho inserito delle Media Queries in fondo al file CSS. Sotto la soglia dei 768px (tablet e telefoni), il menù orizzontale in alto sparisce e viene sostituito dal classico “Menu ad hamburger”. Inoltre, sugli schermi ancora più piccoli (sotto i 480px), ho fatto in modo che i tabelloni per inserire i risultati occupassero il 100% della larghezza disponibile, ingrandendo un po’ i campi di input per rendere più facileappare i numeri con il dito:

```

1 @media (max-width: 480px) {
2   .profile-container {
3     padding: 25px 15px;
4     margin: 20px 10px;
5   }
6   #boxInserimentoRisultati, .match-box {
7     width: 100%;
8     padding: 20px 12px;
9     box-sizing: border-box;
10  }
11  .select-game {
12    width: 22%;
13    height: 46px ;
14    font-size: 1.1em;
15  }
16 }

```

Listing 5.2: Le mie media query per gli smartphone a 480px

6.1 Popolare le pagine senza ricaricarle (Fetch API)

Volevo che l'applicazione fosse molto reattiva. Invece di far generare le pagine HTML dal server ho tenuto l'HTML separato dal backend e usato JavaScript lato client per compilare le pagine.

Ad esempio, quando un utente apre `ranking.html`, la pagina inizialmente è vuota. Non appena il browser ha finito di caricare il DOM, scatta un evento in JavaScript che chiama la funzione `fetch('/api/ranking')`. Il server risponde inviando un file JSON con l'elenco dei giocatori. A quel punto, il mio JavaScript prende questo array, fa un `.map()` per generare delle righe della tabella e le “inietta” dentro l'HTML. In questo modo l'esperienza utente è fluida.

6.2 L'uso di Google Charts

Per rendere la pagina del profilo più carina, ho pensato di aggiungere un grafico a torta che mostrasse visivamente il rapporto tra partite vinte e perse. Ho integrato le API gratuite di Google Charts. Una volta che la chiamata asincrona al mio server mi restituisce il numero di vittorie e sconfitte, passo queste variabili a una funzione che disegna il grafico in un tag `<div>` specifico (`#winLossChart`).

```
1 function drawChart(vinte, perse) {
2   if (vinte === 0 && perse === 0) return;
3   const data = google.visualization.arrayToDataTable([
4     ['Esito', 'Match'],
5     ['Vinte', vinte],
6     ['Perse', perse]
7   ]);
8   new google.visualization.PieChart(document.getElementById('
winLossChart')).draw(data, {
9     colors: ['#053C26', '#CB5A3E'],
10    pieHole: 0.4,
11    backgroundColor: 'transparent'
12  });
13 }
```

Listing 6.1: La funzione usata per disegnare il grafico a torta

7.1 Il problema con la Specificità del CSS

Durante lo sviluppo della pagina del profilo, volevo che il form per inserire i risultati delle partite fosse inizialmente nascosto e apparisse solo cliccando su un bottone. Nel mio file CSS, avevo dato al form un ID (`#boxInserimentoRisultati`) e gli avevo assegnato `display: none;`. Poi in JavaScript, al click del bottone, aggiungevo al form la classe `.is-visible` che avevo impostato su `display: block;`.

Il problema è che il form non compariva mai! Facendo un po' di debugging con gli strumenti di Chrome, ho capito che era un problema di **specificità CSS**. Nel CSS, un selettore basato su ID “vince” sempre su un selettore di classe. Quindi il `display: none` dell'ID era più forte del `display: block` della classe.

7.2 Le modifiche alle partite

L'ostacolo più grande dal punto di vista logico è stato nel backend, quando ho dovuto implementare la funzione per permettere agli amministratori di modificare il risultato di un match passato.

Se l'amministratore modifica i set e l'esito finale della partita cambia (cioè chi aveva vinto diventa il perdente), io non potevo semplicemente aggiornare i set nella tabella **Partite**. Dovevo anche ricalcolare la classifica in modo retroattivo! Il ragionamento che ho dovuto tradurre in codice è stato:

1. Capire chi era il *vecchio* vincitore e togliergli i punti accumulati (-10), abbassare le sue “partite vinte” e alzare le sue “partite perse”.
2. Identificare il *nuovo* vincitore (in base ai set corretti) e assegnargli i punti e la vittoria.
3. Aggiornare i numeri effettivi dei set nel database.

Tutte queste query sul database (**UPDATE**) dovevano essere fatte in successione. Mi sono accorto che se la connessione cadeva nel bel mezzo di queste query, il database restava in uno stato “incoerente”. Quindi, nel file `routes/match.js`, ho racchiuso tutte queste operazioni asincrone (`await dbPromise.query(...)`) all'interno di un blocco `try/catch`. In caso di errore, il processo si interrompe e il server manda un messaggio di errore al client, preservando (nei limiti) la stabilità della classifica.

Per testare l'applicazione sul proprio computer locale, bisogna seguire questa piccola guida che ho preparato:

8.1 Cosa serve avere installato

- **Node.js**. Serve per far girare il server backend.
- Un database **MySQL** attivo in locale.

8.2 Preparare il Database

Puoi configurare il database tramite un'interfaccia grafica oppure seguendo i passaggi attraverso la riga di comando.

Tramite Terminale

1. Apri il terminale e accedere alla shell di MySQL come amministratore digitando:
`sudo mysql -u root -p`
2. Crea il database vuoto digitando:
`CREATE DATABASE circolo_tennis;`
3. Seleziona il database appena creato per indicare a MySQL di operare al suo interno:
`USE circolo_tennis;`
4. Crea l'utente dedicato con la relativa password:
`CREATE USER 'admin_tennis'@'localhost' IDENTIFIED BY 'Pippo04!';`
5. Assegna al nuovo utente tutti i permessi necessari per operare sul database:
`GRANT ALL PRIVILEGES ON circolo_tennis.* TO 'admin_tennis'@'localhost';`
6. Rendi attive le modifiche ai permessi appena assegnati:
`FLUSH PRIVILEGES;`
7. Importa la struttura delle tabelle caricando il file SQL presente nel progetto. Assicurarsi di inserire il percorso assoluto corretto del proprio computer:
`SOURCE ../../WebTennis/setup.sql;`

8. Inserisci l'account amministratore di default all'interno del database:

```
INSERT INTO Utenti (nome, cognome, email, password, genere, is_admin) VALUES ('Admin', 'Admin', 'admin', 'admin', 'M', 1);
```
9. Esci dalla shell di MySQL digitando `exit`.
10. Apri il file `app.js` alla riga 17 per verificare o aggiornare le credenziali del *Connection Pool* affinché combacino con quelle appena configurate.

8.3 Avviare il server

1. apri il terminale (o il prompt dei comandi) dentro la cartella radice del progetto WebTennis.
2. Prima bisogna scaricare le librerie necessarie (Express, mysql2, jsonwebtoken, cookie-parser). Per farlo, scrivi il comando:

```
1 npm install express mysql2 jsonwebtoken cookie-parser
2
```

3. Una volta finita l'installazione, fai partire il server con:

```
1 npm run dev
2
```

4. Se tutto è andato bene, vedrai stampato nel terminale “Server attivo su `http://localhost:3000`” e “Connesso al database MySQL con successo”. A questo punto apri il browser a quell'indirizzo.

In quest'ultimo capitolo riporto il codice sorgente dei file più importanti dell'applicazione, per mostrare concretamente il lavoro fatto.

9.1 Il Backend: app.js

Questo è il file principale dell'applicativo Node.js. All'inizio importo le librerie, poi configuro la connessione a MySQL. Subito dopo, configuro Express in modo che serva i file della cartella `public` liberamente, ma blocco la visualizzazione dei file nella cartella `private` usando la mia funzione middleware `verificaAutenticazionePagine`. Infine, attacco le rotte delle API.

```
1 const express = require('express');
2 const mysql = require('mysql2');
3 const path = require('path');
4 const cookieParser = require('cookie-parser');
5
6 const {
7   verificaAutenticazioneAPI,
8   verificaAutenticazionePagine
9 } = require('./middleware/auth');
10
11 const app = express();
12 const PORT = 3000;
13
14 // Configurazione Connessione MySQL
15 const db = mysql.createConnection({
16   host: "localhost",
17   user: "admin_tennis",
18   password: "Pippo04!",
19   database: "circolo_tennis",
20   waitForConnections: true
21 });
22
23 db.connect((err) => {
24   if (err) return console.error('Errore database:', err);
25   console.log('Connesso al database MySQL con successo.');
```



```

34     verificaAutenticazionePagine,
35     (req, res) => {
36         res.sendFile(path.join(__dirname, 'private', req.path));
37     }
38 );
39
40 // Pagine pubbliche e script accessibili a tutti
41 app.use(express.static(path.join(__dirname, 'public')));
42 app.use('/script', express.static(path.join(__dirname, 'script')));
43
44 // Smistamento delle chiamate API
45 app.use('/api', require('./routes/auth')(db));
46 app.use('/api', require('./routes/player')(db, verificaAutenticazioneAPI));
47 app.use('/api', require('./routes/match')(db, verificaAutenticazioneAPI));
48
49 // Se si digita un URL sbagliato, mostro la pagina 404
50 app.use((req, res) => {
51     res.status(404).sendFile(path.join(__dirname, 'public', '404.html'));
52 });
53
54 app.listen(PORT, () => {
55     console.log('Server attivo su http://localhost:${PORT}');
56 });

```

Listing 9.1: app.js - Avvio server e routing

9.2 La Sicurezza: middleware/auth.js

Qui c'è il codice che verifica se un utente ha il permesso o meno di accedere. Uso la libreria `jsonwebtoken` per decodificare il cookie. Ho dovuto creare due funzioni distinte, perché se blocco l'accesso a una pagina HTML voglio rimandare l'utente alla schermata `denied.html` (facendo `res.redirect()`), mentre se blocco una chiamata API fatta in background devo restituire un JSON con l'errore.

```

1  const jwt = require('jsonwebtoken');
2  // La chiave segreta per firmare i token. In produzione andrebbe in un
   file .env!
3  const JWT_SECRET = 'esame_prog_web_circolo_tennis_secret';
4
5  // Protezione delle chiamate API (JSON)
6  function verificaAutenticazioneAPI(req, res, next) {
7      const token = req.cookies.token;
8      if (!token) {
9          return res.status(401).json({ error: 'Accesso negato.' });
10     }
11     try {
12         req.user = jwt.verify(token, JWT_SECRET);
13         next();
14     } catch (err) {
15         res.clearCookie('token');
16         return res.status(401).json({ error: 'Sessione non valida.' });
17     }
18 }

```

```

19
20 // Protezione delle rotte HTML (Redirect)
21 function verificaAutenticazionePagine(req, res, next) {
22     const token = req.cookies.token;
23     if (!token) {
24         return res.redirect('/denied.html');
25     }
26     try {
27         req.user = jwt.verify(token, JWT_SECRET);
28         next();
29     } catch (err) {
30         res.clearCookie('token');
31         return res.redirect('/denied.html');
32     }
33 }
34
35 module.exports = {
36     JWT_SECRET,
37     verificaAutenticazioneAPI,
38     verificaAutenticazionePagine
39 };

```

Listing 9.2: middleware/auth.js - Filtro per i cookie

9.3 Login e Registrazione: routes/auth.js

In questo file gestisco la creazione di nuovi account e l'emissione del JWT al login. Una particolarità è che, durante la registrazione, inserisco il nome e l'email dell'utente, ma poi eseguo subito un'altra query per generargli una riga a punteggio zero nella tabella Classifiche. In questo modo il giocatore appare subito in fondo alla graduatoria.

```

1 const express = require('express');
2 const jwt = require('jsonwebtoken');
3 const JWT_SECRET = 'esame_prog_web_circolo_tennis_secret';
4
5 module.exports = (db) => {
6     const router = express.Router();
7     const dbPromise = db.promise();
8
9     // Registrazione di un nuovo socio
10    router.post('/register', async (req, res) => {
11        const { nome, cognome, email, password, genere } = req.body;
12        if (!nome || !cognome || !email || !password || !genere) {
13            return res.status(400).json({ error: 'Campi obbligatori mancanti.' });
14        }
15
16        try {
17            // Creo l'utente (is_admin e' sempre 0 di base)
18            const [result] = await dbPromise.query(
19                'INSERT INTO Utenti (nome, cognome, email, password, genere, is_admin) VALUES (?, ?, ?, ?, ?, 0)',
20                [nome, cognome, email, password, genere]
21            );
22
23            // Genero la sua riga vuota in classifica associandogli il suo nuovo ID

```

```

24         await dbPromise.query(
25             'INSERT INTO Classifiche (id_giocatore,
punteggio_attuale, livello, partite_vinte, partite_perse) VALUES (?,
0, 'Principiante', 0, 0)',
26             [result.insertId]
27         );
28
29         res.status(201).json({ message: 'Socio registrato!' });
30     } catch (err) {
31         console.error(err);
32         // Se MySQL si lamenta che la mail esiste gia', gestisco l'
errore
33         if (err.code === 'ER_DUP_ENTRY') {
34             return res.status(400).json({ error: 'Email gia in uso.'
});
35         }
36         return res.status(500).json({ error: err.message });
37     }
38 });
39
40 // Login del socio
41 router.post('/login', (req, res) => {
42     const { nome, password } = req.body;
43     db.query(
44         'SELECT * FROM Utenti WHERE nome = ? AND password = ?',
45         [nome, password],
46         (err, results) => {
47             if (err) return res.status(500).json({ error: err.
message });
48             if (results.length === 0) return res.status(401).json({
error: 'Credenziali errate.' });
49
50             const utente = results[0];
51
52             // Genero il "Gettone" JWT
53             const token = jwt.sign(
54                 {userId: utente.id_utente, isAdmin: utente.is_admin
},
55                 JWT_SECRET,
56                 { expiresIn: '1h' }
57             );
58
59             // Mando il gettone al client dentro un cookie sicuro
60             res.cookie('token', token, {
61                 httpOnly: true,
62                 sameSite: 'strict',
63                 maxAge: 3600000 // dura 1 ora
64             });
65
66             res.json({
67                 message: 'Accesso eseguito!',
68                 utente: { id_utente: utente.id_utente, nome: utente.
nome, cognome: utente.cognome, is_admin: utente.is_admin }
69             });
70         }
71     );
72 });
73

```

```

74     return router;
75 };

```

Listing 9.3: routes/auth.js - Gestione degli utenti

9.4 La logica dei Punteggi: routes/match.js

Qui è dove implemento la logica per decretare chi vince un match inserito dai giocatori. In base a chi vince due set su tre, il backend stabilisce l'ID del vincitore e va ad assegnargli 10 punti extra nel database.

```

1  const express = require('express');
2
3  module.exports = (db, verificaAutenticazioneAPI) => {
4      const router = express.Router();
5      const dbPromise = db.promise();
6
7      // Funzioncina utile per capire chi ha vinto la partita contando i
      // set
8      const calcolaVincitore = (id1, id2, s1_1, s1_2, s2_1, s2_2, s3_1,
      s3_2) => {
9          let set1 = 0, set2 = 0;
10         if (parseInt(s1_1) > parseInt(s1_2)) set1++; else set2++;
11         if (parseInt(s2_1) > parseInt(s2_2)) set1++; else set2++;
12         if (s3_1 && s3_2) { // Il terzo set si conta solo se e' stato
      // giocato
13             if (parseInt(s3_1) > parseInt(s3_2)) set1++; else set2++;
14         }
15         return set1 > set2 ? id1 : id2;
16     };
17
18     // Inserimento di un nuovo Match
19     router.post('/match', verificaAutenticazioneAPI, async (req, res) => {
20         const { id_avversario, set1_g1, set1_g2, set2_g1, set2_g2,
      set3_g1, set3_g2 } = req.body;
21         const id_utente = req.user.userId;
22
23         if (!id_avversario || set1_g1 === undefined || set1_g2 ===
      undefined) {
24             return res.status(400).json({ error: "Dati incompleti." });
25         }
26
27         const id_vincitore = calcolaVincitore(id_utente, id_avversario,
      set1_g1, set1_g2, set2_g1, set2_g2, set3_g1, set3_g2);
28         const id_perdente = id_vincitore === id_utente ? id_avversario :
      id_utente;
29
30         try {
31             await dbPromise.query(
32                 `INSERT INTO Partite (id_giocatore1, id_giocatore2,
      set1_g1, set1_g2, set2_g1, set2_g2, set3_g1, set3_g2, id_vincitore)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
33                 [id_utente, id_avversario, set1_g1, set1_g2, set2_g1,
      set2_g2, set3_g1 || null, set3_g2 || null, id_vincitore]
34             );
35         }

```

```

36         // Do i 10 punti in premio al vincitore
37         await dbPromise.query('UPDATE Classifiche SET
punteggio_attuale = punteggio_attuale + 10, partite_vinte =
partite_vinte + 1 WHERE id_giocatore = ?', [id_vincitore]);
38
39         // Aggiorno le statistiche dello sconfitto
40         await dbPromise.query('UPDATE Classifiche SET partite_perse
= partite_perse + 1 WHERE id_giocatore = ?', [id_perdente]);
41
42         res.json({ message: "Match registrato con successo!" });
43     } catch (err) {
44         console.error(err);
45         return res.status(500).json({ error: "Errore salvataggio: "
+ err.message });
46     }
47 });
48
49 return router;
50 };

```

Listing 9.4: routes/match.js - L'intelligenza dietro le partite

9.5 Il File Frontend: script/giocatore.js

Infine, presento il codice JavaScript che gira nel browser sulla pagina del profilo utente. È sicuramente il file client-side più elaborato che ho scritto. L'ho progettato in modo che faccia una serie di `fetch` asincrone per:

1. Disegnare il grafico a torta.
2. Caricare lo storico delle partite passate dell'utente.
3. Se riconosce che siamo amministratori, trasforma al volo (modificando dinamicamente il DOM dell'HTML) la card dello storico in un mini-form di modifica per alterare il risultato della partita.

```

1 // Preparo le API dei grafici
2 google.charts.load('current', { packages: ['corechart'] });
3
4 let utenteAttuale = null;
5 let idProfilo = null;
6 const partiteCache = {}; // Mi salvo i match caricati per non doverli
    richiedere al database
7
8 const $ = id => document.getElementById(id);
9 const isAdmin = () => utenteAttuale && utenteAttuale.is_admin === 1;
10
11 // 1. LEGGI I PUNTEGGI (mi serve sia per inserire che per modificare)
12 function leggiPunteggi(idPartita = null) {
13     // Se c'è idPartita legge dai campi del form di modifica,
    altrimenti dal normale form
14     const s1g1 = idPartita ? $('edit_s1_1_${idPartita}').value : $('
g1_mio').value;
15     const s1g2 = idPartita ? $('edit_s1_2_${idPartita}').value : $('
g1_lui').value;

```

```

16     const s2g1 = idPartita ? $('edit_s2_1_${idPartita}').value : $('
17     g2_mio').value;
17     const s2g2 = idPartita ? $('edit_s2_2_${idPartita}').value : $('
18     g2_lui').value;
18     const s3g1 = idPartita ? $('edit_s3_1_${idPartita}').value : $('
19     g3_mio').value;
19     const s3g2 = idPartita ? $('edit_s3_2_${idPartita}').value : $('
20     g3_lui').value;
21
22     if (s1g1 === '' || s1g2 === '' || s2g1 === '' || s2g2 === '') {
23         return null;
24     }
25
26     return {
27         set1_g1: parseInt(s1g1),
28         set1_g2: parseInt(s1g2),
29         set2_g1: parseInt(s2g1),
30         set2_g2: parseInt(s2g2),
31         set3_g1: s3g1 === '' ? null : parseInt(s3g1),
32         set3_g2: s3g2 === '' ? null : parseInt(s3g2)
33     };
34 }
35
36 // 2. VERIFICA SE IL MATCH E' VALIDO
37 function matchValido(p) {
38     let setVintiG1 = 0;
39     let setVintiG2 = 0;
40
41     if (p.set1_g1 > p.set1_g2) setVintiG1++; else setVintiG2++;
42     if (p.set2_g1 > p.set2_g2) setVintiG1++; else setVintiG2++;
43     if (p.set3_g1 !== null && p.set3_g2 !== null) {
44         if (p.set3_g1 > p.set3_g2) setVintiG1++; else setVintiG2++;
45     }
46
47     return setVintiG1 !== setVintiG2;
48 }
49
50 // 3. RECUPERO I DATI DELL'UTENTE
51 async function caricaProfilo() {
52     const urlCompleto = window.location.href;
53     const partiUrl = urlCompleto.split('?');
54     const parametri = partiUrl[1].split('=');
55     idProfilo = parseInt(parametri[1]);
56     if (!idProfilo) return;
57
58     try {
59         const resMe = await fetch('/api/me');
60         if (resMe.ok) utenteAttuale = await resMe.json();
61
62         // Chiedo le info del giocatore al backend
63         const giocatore = await fetch('/api/giocatore/${idProfilo}').
64         then(r => r.json());
65
66         // Modifico l'HTML al volo coi dati ricevuti
67         $('playerTitle').innerText = `${giocatore.nome} ${giocatore.
68         cognome}`;
69         $('playerLevel').innerText = `Livello: ${giocatore.livello}`;
70         $('statPunti').innerText = giocatore.punteggio_attuale;

```

```

68     $('statVinte').innerText = giocatore.partite_vinte;
69     $('statPerse').innerText = giocatore.partite_perse;
70
71     google.charts.setOnLoadCallback(() => drawChart(giocatore.
partite_vinte, giocatore.partite_perse));
72
73     if (isAdmin() || utenteAttuale.id_utente === idProfilo) {
74         await preparaFormInserimento();
75     }
76
77     await caricaStoricoPartite();
78
79     } catch (err) {
80         console.error("Errore nel caricamento del profilo:", err);
81     }
82 }
83
84 // 4. CARICA LO STORICO DEI MATCH E ABILITA I TASTI ADMIN
85 async function caricaStoricoPartite() {
86     const lista = $('listaPartite');
87     if (!lista) return;
88
89     try {
90         const response = await fetch(`/api/giocatore/${idProfilo}/
partite`);
91         const partite = await response.json();
92
93         if (partite.length === 0) {
94             lista.innerHTML = '<p class="text-italic">Non hai ancora
giocato nessuna partita.</p>';
95             return;
96         }
97
98         // Faccio un loop (map) sull'array delle partite per generare le
card HTML
99         lista.innerHTML = partite.map(p => {
100             partiteCache[p.id_partita] = p; // salvo nella cache
101
102             const vittoria = p.id_vincitore === idProfilo;
103             let punteggioStr = `${p.set1_g1}-${p.set1_g2}, ${p.set2_g1}-
${p.set2_g2}`;
104             if (p.set3_g1 !== null && p.set3_g2 !== null) {
105                 punteggioStr += ', ${p.set3_g1}-${p.set3_g2}';
106             }
107
108             const dataFormatted = new Date(p.data_match).
toLocaleDateString('it-IT', {
109                 day: '2-digit', month: '2-digit', year: 'numeric', hour:
'2-digit', minute: '2-digit'
110             });
111
112             // Se chi sta guardando e' admin, aggiungo il bottone di
modifica all'HTML
113             const adminButton = isAdmin() ? '
114                 <div class="admin-edit-controls" style="margin-top: 10px
; ">
115                     <button class="btn-modifica-match" onclick="
mostraFormModifica(${p.id_partita})">Modifica Punteggio</button>

```

```

116         </div>
117         ' : ' ';
118
119         return '
120         <div class="match-history-card" id="match-card-#{p.
121 id_partita}">
122             <div class="match-card-row">
123                 <div>
124                     <span class="match-esito-badge #{vittoria ?
125 'esito-vittoria' : 'esito-sconfitta'}">
126                         #{vittoria ? 'Vittoria' : 'Sconfitta'}
127                     </span>
128                     <span class="match-title">#{p.nome_g1} #{p.
129 cognome_g1} vs #{p.nome_g2} #{p.cognome_g2}</span>
130                 </div>
131                 <span class="match-date">#{dataFormatted}</span>
132             </div>
133             <div class="match-card-body">
134                 Punteggio: <strong class="match-score">#{
135 punteggioStr}</strong>
136             </div>
137             #{adminButton}
138         </div>
139         ';
140     }).join('');
141 } catch (err) {
142     console.error(err);
143     lista.innerHTML = '<p class="msg error">Impossibile scaricare lo
144 storico dal server.</p>';
145 }
146 }
147
148 // 5. TRASFORMA LA CARD DEL MATCH IN UN FORM (Inline Edit)
149 function mostraFormModifica(idPartita) {
150     const p = partiteCache[idPartita];
151     const card = $('match-card-#{idPartita}');
152
153     const val3_1 = p.set3_g1 !== null ? p.set3_g1 : '';
154     const val3_2 = p.set3_g2 !== null ? p.set3_g2 : '';
155
156     // Sostituisco il blocco visuale con gli input per modificare i
157     numeri!
158     card.innerHTML = '
159     <div class="tennis-scoreboard edit-board-box">
160         <h4 class="edit-board-title">Modifica Punteggio Admin</h4>
161
162         <div class="score-row">
163             <span class="row-label">#{p.nome_g1}</span>
164             <input type="number" id="edit_s1_1_#{idPartita}" class="
165 select-game" min="0" max="7" value="#{p.set1_g1}" required>
166             <input type="number" id="edit_s2_1_#{idPartita}" class="
167 select-game" min="0" max="7" value="#{p.set2_g1}" required>
168             <input type="number" id="edit_s3_1_#{idPartita}" class="
169 select-game" min="0" max="7" value="#{val3_1}" placeholder="-">
170         </div>
171
172         <div class="score-row">

```



```

165         <span class="row-label">${p.nome_g2}</span>
166         <input type="number" id="edit_s1_2_${idPartita}" class="
select-game" min="0" max="7" value="${p.set1_g2}" required>
167         <input type="number" id="edit_s2_2_${idPartita}" class="
select-game" min="0" max="7" value="${p.set2_g2}" required>
168         <input type="number" id="edit_s3_2_${idPartita}" class="
select-game" min="0" max="7" value="${val3_2}" placeholder="-">
169     </div>
170
171     <div class="edit-actions-wrap">
172         <button class="btn-salva btn-salva-modifica" onclick="
salvaModificaMatch(${idPartita})">Salva Nuovi Dati</button>
173         <button class="btn-annulla-modifica" onclick="location.
reload()">Annulla</button>
174     </div>
175     <div id="edit-error-${idPartita}" class="edit-error-msg"></
div>
176 </div>
177     `;
178 }
179
180 // 6. MANDA LA PUT REQUEST AL SERVER
181 async function salvaModificaMatch(idPartita) {
182     const errDiv = $('edit-error-${idPartita}');
183     const punteggi = leggiPunteggi(idPartita);
184
185     if (!punteggi) {
186         errDiv.innerText = "Attenzione: devi compilare i primi due set."
187     };
188     return;
189 }
190
191 if (!matchValido(punteggi)) {
192     errDiv.innerText = "Errore: la partita non puo' finire pari!";
193     return;
194 }
195
196 try {
197     const response = await fetch(`/api/match/${idPartita}`, {
198         method: 'PUT',
199         headers: { 'Content-Type': 'application/json' },
200         body: JSON.stringify({
201             set1_g1: punteggi.set1_g1,
202             set1_g2: punteggi.set1_g2,
203             set2_g1: punteggi.set2_g1,
204             set2_g2: punteggi.set2_g2,
205             set3_g1: punteggi.set3_g1,
206             set3_g2: punteggi.set3_g2
207         })
208     });
209
210     if (response.ok) {
211         location.reload(); // Se e' andata bene, ricarico la pagina
212         per riavere la schermata normale
213     } else {
214         const data = await response.json();
215         errDiv.innerText = data.error || "Qualcosa e' andato storto
lato server.";

```

```

214     }
215   } catch {
216     errDiv.innerText = "Errore di connessione a Internet.";
217   }
218 }
219
220 // 7. POPOLA IL MENU A TENDINA CON GLI AVVERSARI
221 async function preparaFormInserimento() {
222   $('boxInserimentoRisultati').classList.add('is-visible');
223
224   const ranking = await fetch('/api/ranking').then(r => r.json());
225   const soci = (ranking.maschile || []).concat(ranking.femminile ||
226   []);
227
228   // Metto i nomi dei soci nella SELECT dell'HTML
229   $('selectAvversario').innerHTML = soci
230     .filter(s => s.id_utente !== idProfilo)
231     .map(s => '<option value="${s.id_utente}">${s.cognome} ${s.nome
232   }</option>')
233     .join('');
234
235   attivaAscoltoForm();
236 }
237
238 // 8. LISTENER DEL SUBMIT
239 function attivaAscoltoForm() {
240   $('matchForm').onsubmit = async e => {
241     e.preventDefault();
242     const msg = $('matchMsg');
243     const punteggi = leggiPunteggi();
244
245     if (!punteggi || !matchValido(punteggi)) {
246       msg.innerText = 'Compila almeno i primi due set in modo
247     corretto!';
248       return;
249     }
250
251     try {
252       const response = await fetch('/api/match', {
253         method: 'POST',
254         headers: { 'Content-Type': 'application/json' },
255         body: JSON.stringify({
256           id_avversario: parseInt($('selectAvversario').value)
257         },
258         {
259           set1_g1: punteggi.set1_g1,
260           set1_g2: punteggi.set1_g2,
261           set2_g1: punteggi.set2_g1,
262           set2_g2: punteggi.set2_g2,
263           set3_g1: punteggi.set3_g1,
264           set3_g2: punteggi.set3_g2
265         })
266       });
267
268       if (response.ok) {
269         msg.innerText = 'Fantastico! Match salvato.';
270         setTimeout(() => location.reload(), 1200);
271       } else {
272         const data = await response.json();

```

```

268         msg.innerText = data.error || 'Server error.';
269     }
270 } catch {
271     msg.innerText = 'Ops, il server non risponde.';
272 }
273 };
274 }
275
276 // 9. LA MAGIA DI GOOGLE CHARTS
277 function drawChart(vinte, perse) {
278     if (vinte === 0 && perse === 0) return;
279
280     const data = google.visualization.arrayToDataTable([
281         ['Esito', 'Match'],
282         ['Vinte', vinte],
283         ['Perse', perse]
284     ]);
285
286     new google.visualization.PieChart($('winLossChart')).draw(data, {
287         colors: ['#053C26', '#CB5A3E'],
288         pieHole: 0.4,
289         backgroundColor: 'transparent'
290     });
291 }
292
293 // Tutto parte da qui quando la pagina ha finito di caricarsi
294 window.addEventListener('DOMContentLoaded', caricaProfilo);

```

Listing 9.5: script/giocatore.js - Tutto il comportamento lato browser del profilo